

Lecture 11

Recurrent Neural Networks

Elena Raponi | Neural Computing course



Universiteit
Leiden
The Netherlands

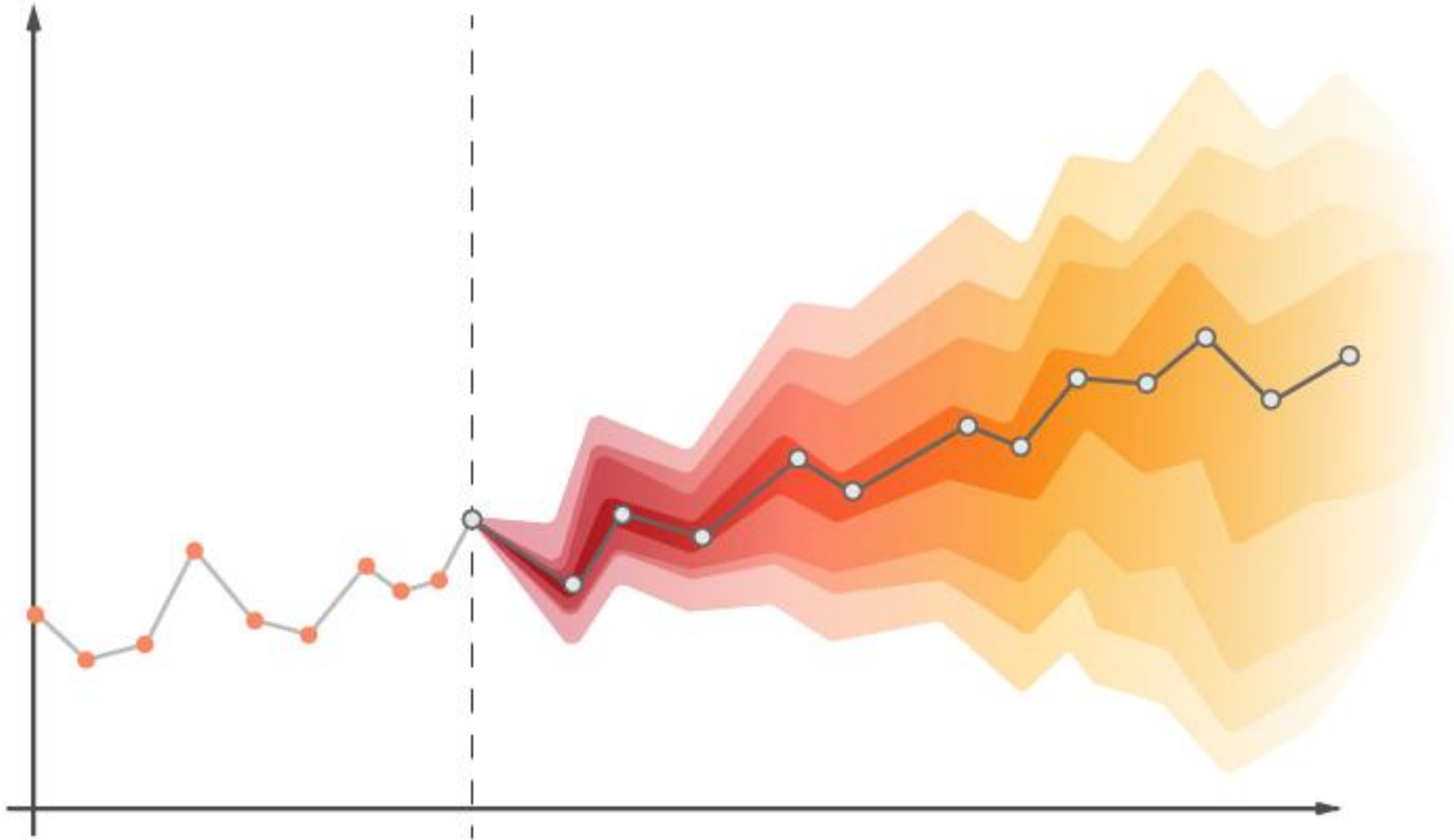
Discover the world at Leiden University

Index

1. Motivation – Time Series Forecasting
2. Introduction
3. Back to Hopfield Networks
4. Vanilla Recurrent Neural Network
5. Sentiment Classification
6. Training
7. Long Short-term Memory Networks
8. Seq2seq models

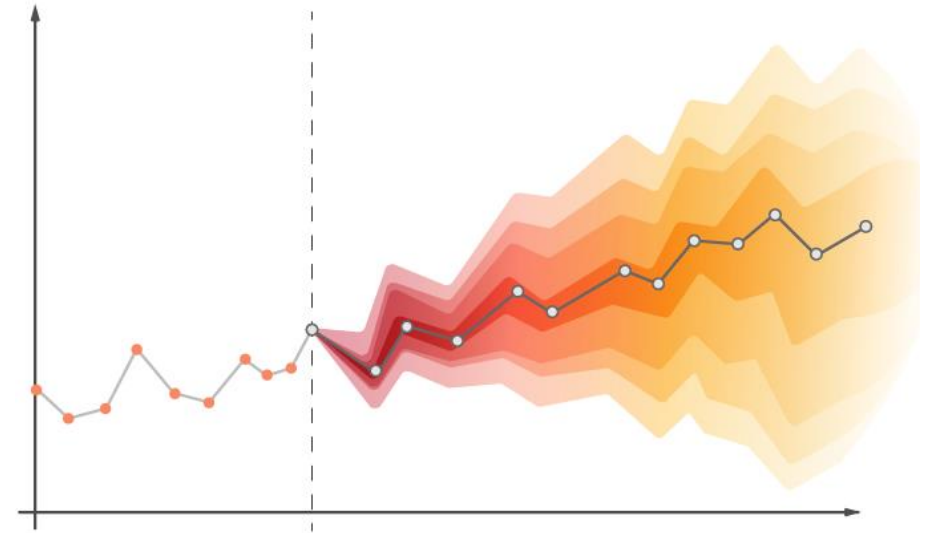


Problem – Time Series Forecasting



Problem – Time Series Forecasting

- Fact:
 - Data comes in a sequence
 - Data point *has temporal dependency*
- Can we solve this task with a multilayer NN?

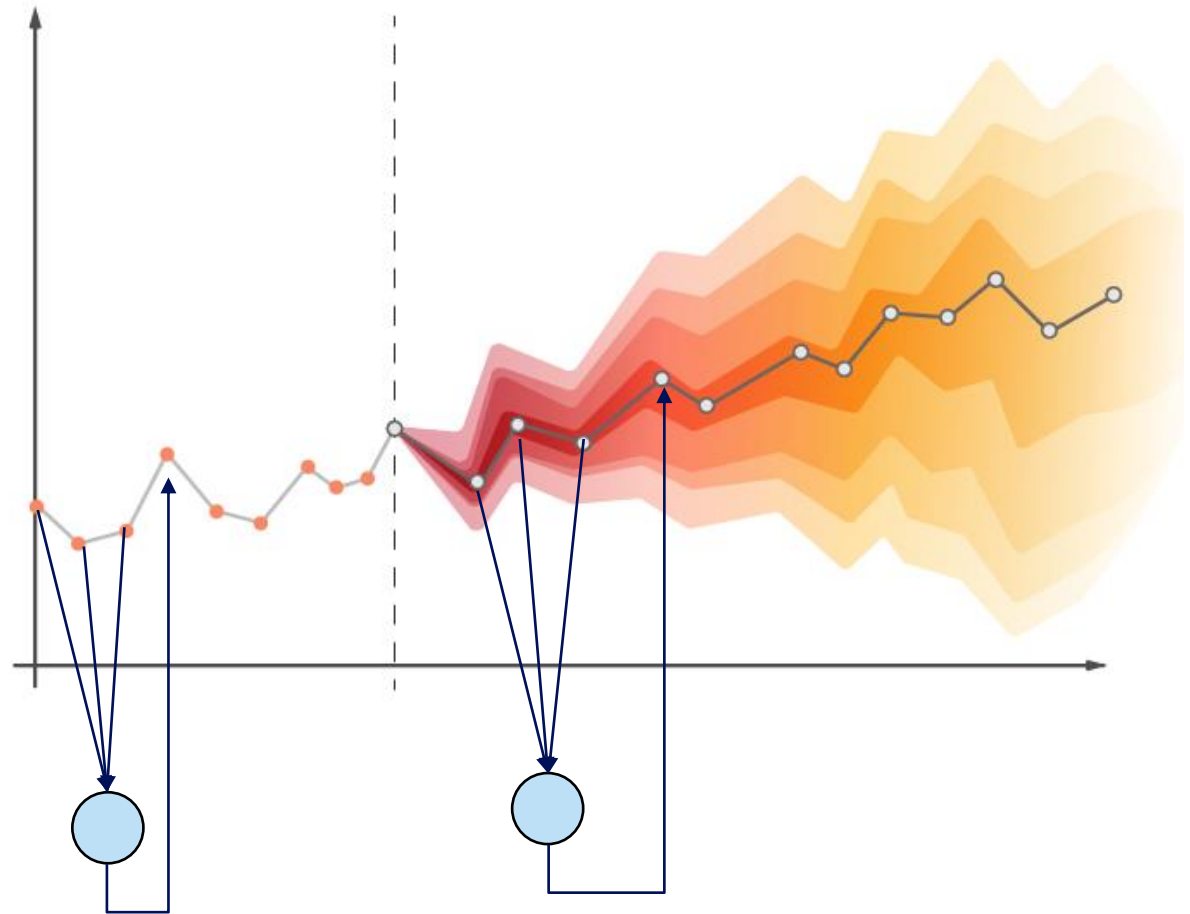


The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .
The FBI is chasing a criminal on the run .

Image source: [Cheng et al., Long Short-Term Memory-Networks for Machine Reading, 2016](#)

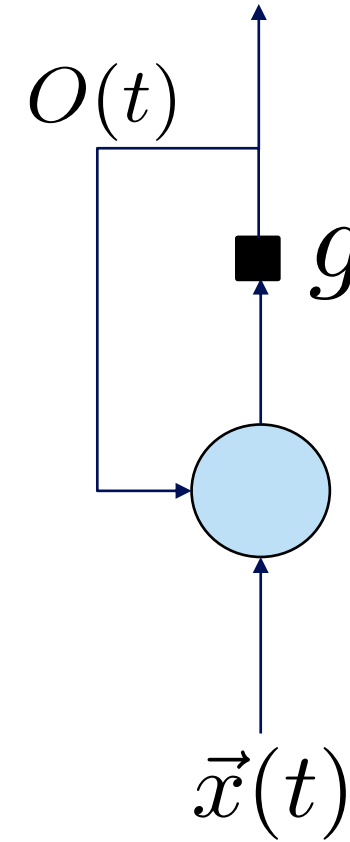
Problem – Time Series Forecasting

- Can we solve this task with a multilayer NN?
- Issues?
 - How long is the past to use?
 - Temporal dependency among the inputs are ignored
 - Input-output relation may also vary through time...



Introduction

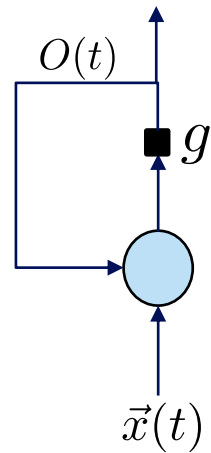
- Recall the characteristic of Multi-layer NN – No cycle
- What if we have cycles in the network?



Introduction

- What if we have cycles in the network?
 - The output of the neuron evolves through time
 - Given a fixed input, it generates a time series as output

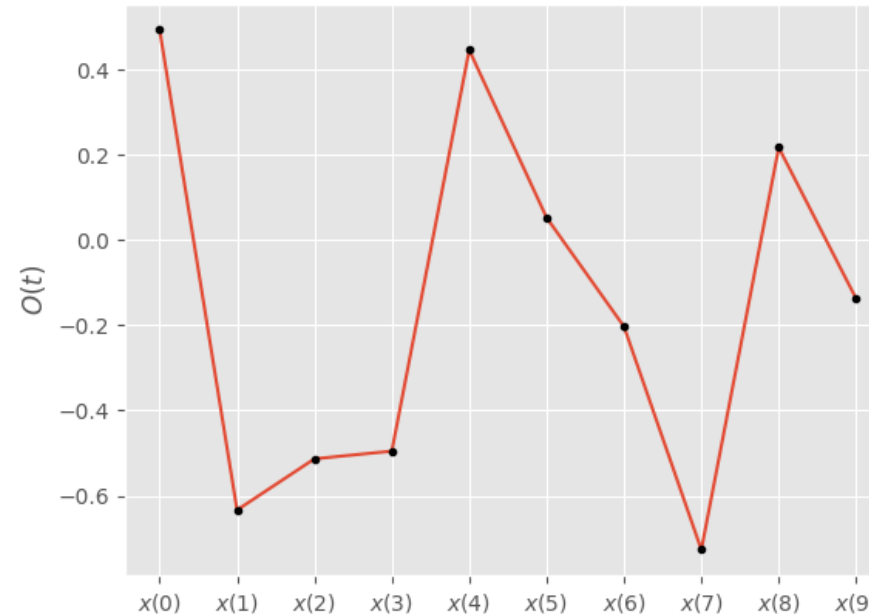
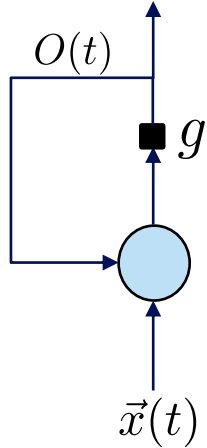
$$O(t) = g(\vec{w}_1^\top \vec{x}(t) + w_2 O(t-1) + b)$$



Introduction

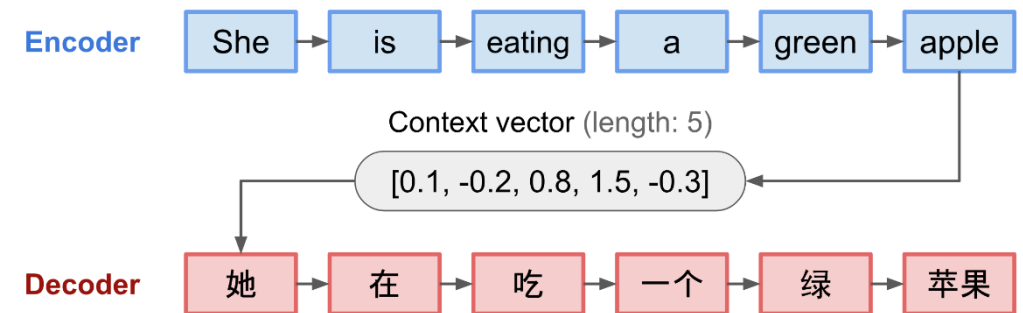
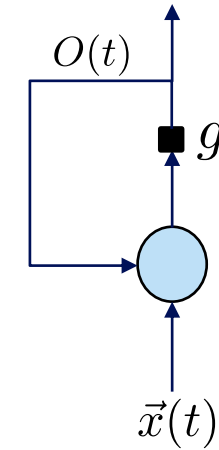
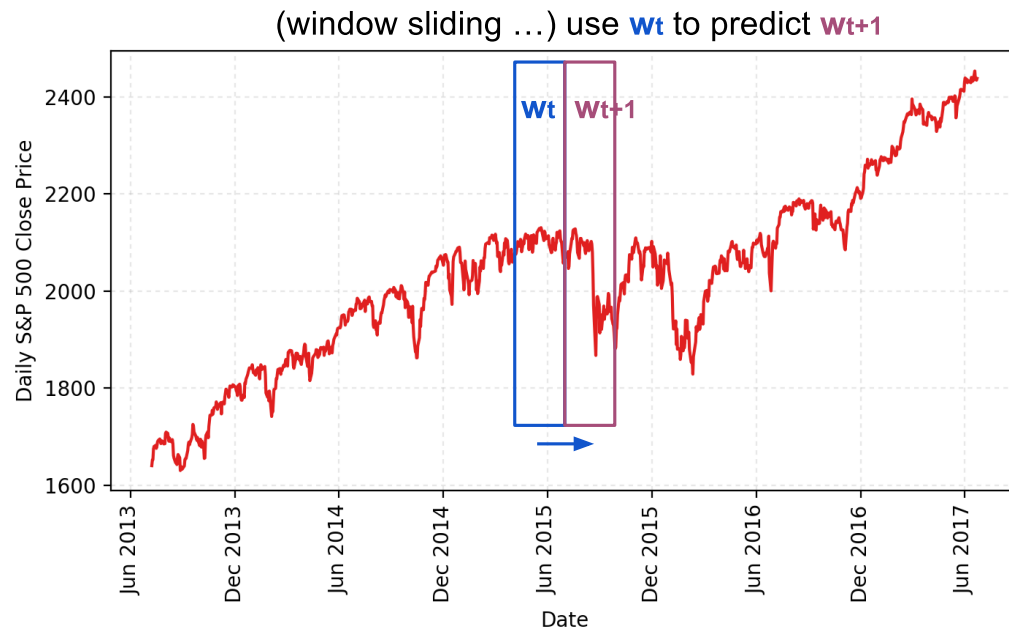
- The output of the neuron evolves through time
- Given a fixed input, it generates a time series as output

$$O(t) = \frac{2}{1 + \exp(-O(t-1) + \mathcal{N}(0, 1))} - 1$$



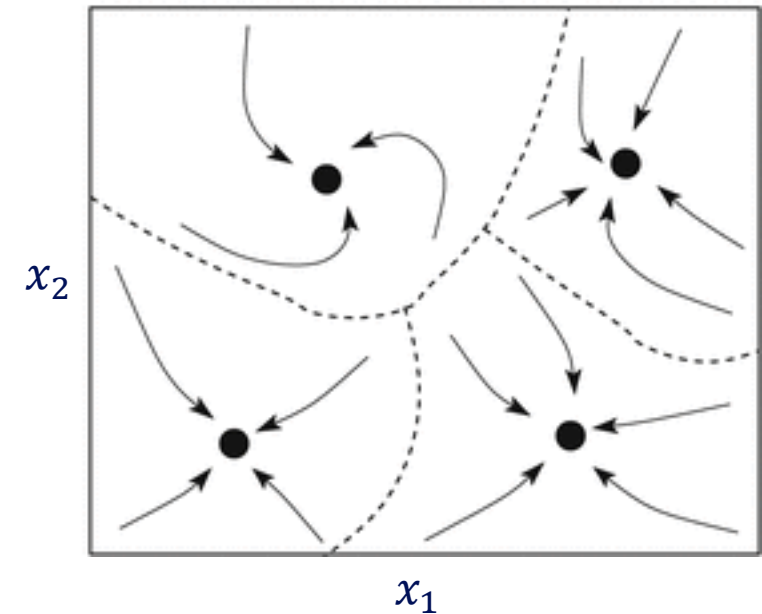
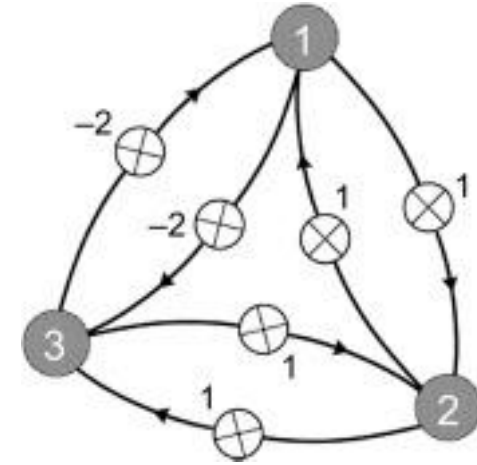
Introduction

- Applications:
 - Memory? – Hopfield Network
 - Time-series? – forecasting, anomaly detection...
 - For language tasks? – machine translation, sentiment analysis...
 - Generative task? - Boltzmann machine



Hopfield Network

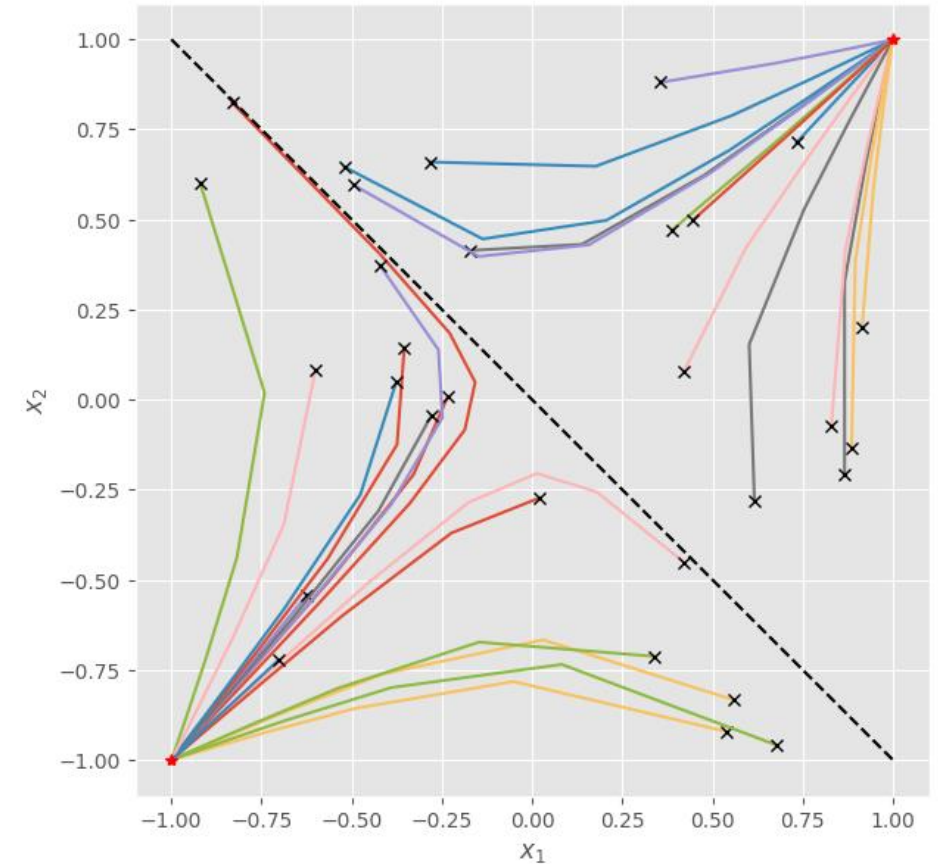
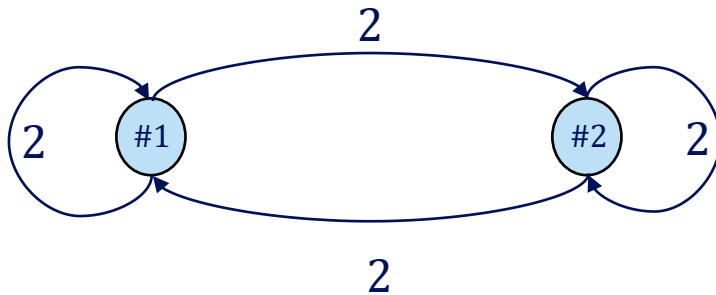
- The model: $O(t + 1) = g \left(\sum_i w_{ij} O_j(t) + b_i \right)$
- g can be tanh or sigmoid
- Given an input, the model will evolve into a ***stable state***
- The stable states partition the input into attracting basins/regions.



Hopfield Network

An example: $O_1(t+1) = \tanh(2 \cdot O_1(t) + 2 \cdot O_2(t))$
 $O_2(t+1) = \tanh(2 \cdot O_1(t) + 2 \cdot O_2(t))$

- Two inputs in the range $[-1, 1]$
- tanh activation; no bias term
- Evolve for 10 steps
- The model:

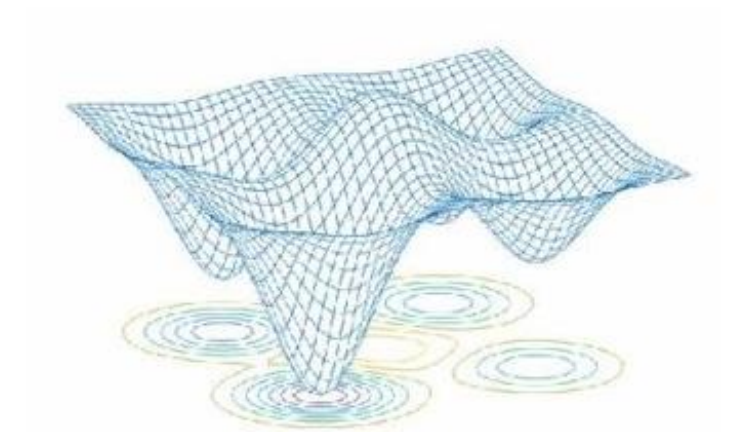


Hopfield Network

- The model: $O(t + 1) = g \left(\sum_i w_{ij} O_j(t) + b_i \right)$
- **Stable states** are the minimizer of the energy function:

$$E(\vec{x}) = -\frac{1}{2} \vec{x}^\top \mathbf{W} \vec{x} - \vec{x}^\top \vec{b}$$

- Automatic gradient descent!

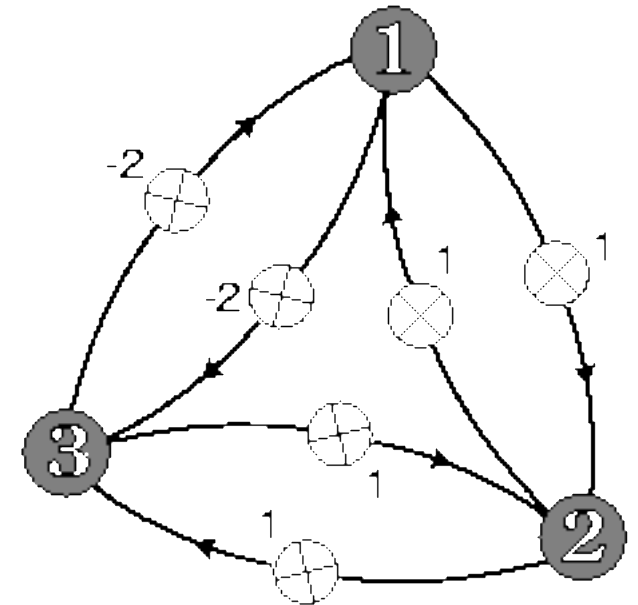


Hopfield Network – discrete version

- We restrict the output to $\{-1, 1\}$:

$$O_i(t + 1) = \text{sgn} \left(\sum_j w_{ij} O_j(t) + b_i \right)$$

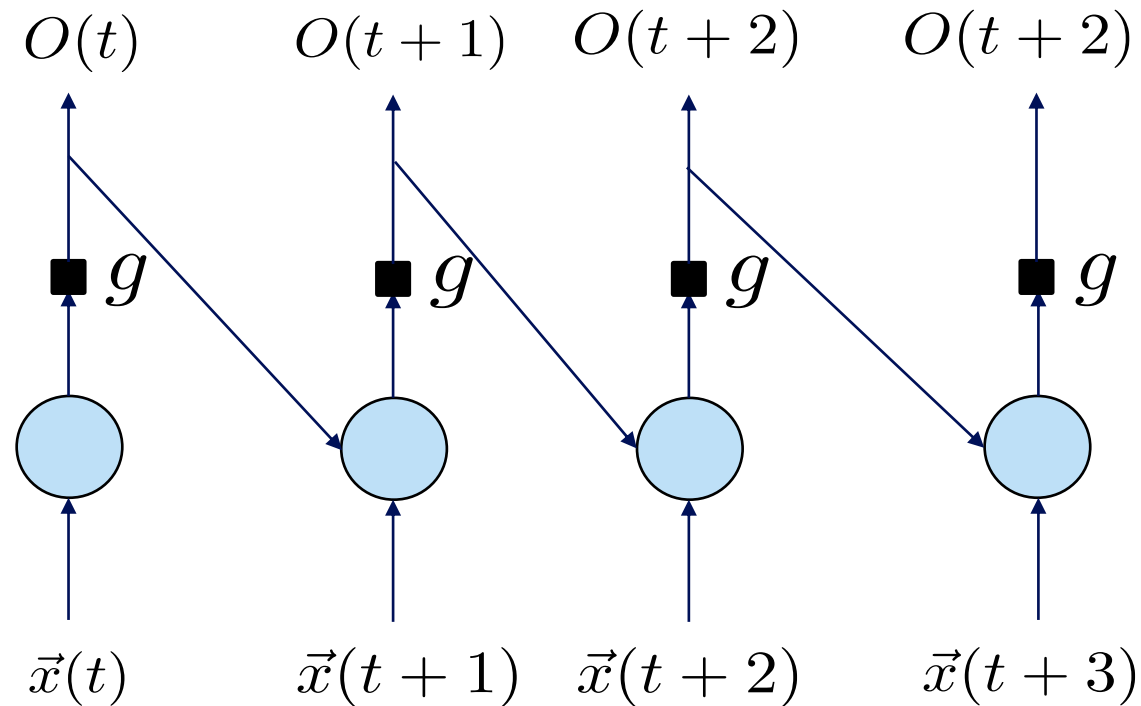
- Associative memory
 - Capable of store a collection of patterns $\{\vec{x}_i\}_{1 \leq i \leq n}$
 - If present a new (not stored) pattern $\vec{x}$, the network will produce the stored pattern most closely resemble $\vec{x}$.
- If we set the weights to:
$$w_{ij} \propto x_i x_j$$
- We will get $\vec{x}$ back from the network



3 node Hopfield net

Computation Graph for RNNs

- **Unfolding** of a simple RNN in time – feedforward structure



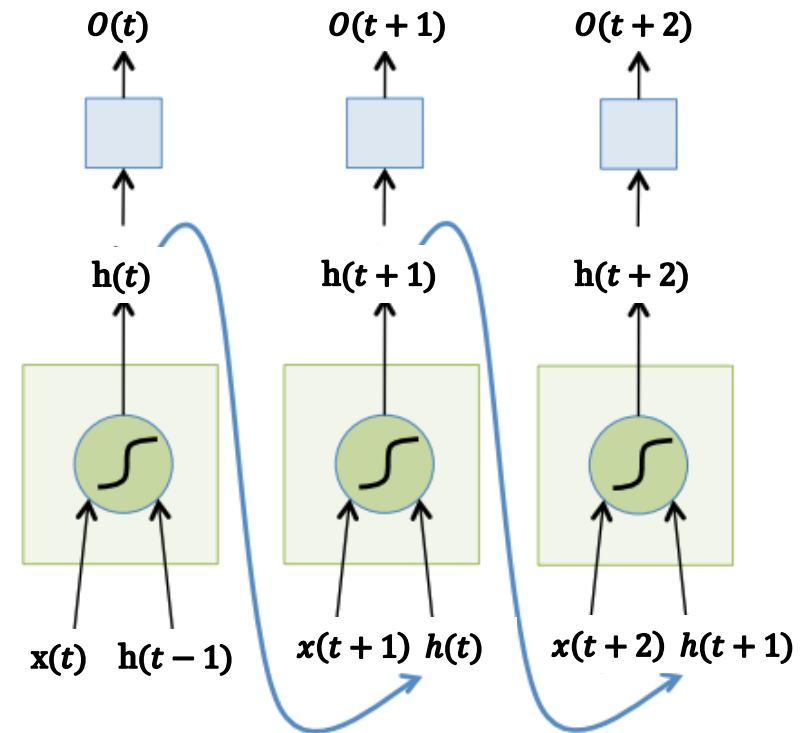
Vanilla RNN

- Introducing the hidden state

- The **internal state** of an RNN cell – represents a short memory of the history
- Different from the output
- Why do we need it?

$$h(t) = \text{sigmoid} \left(W \begin{bmatrix} x(t) \\ h(t-1) \end{bmatrix} \right)$$

$$O(t) = g(h(t))$$



Vanilla RNN cell for simple NLP task

Sentiment Classification

- E.g., reviews on movies from Netflix
- **Input** – words, sentences
- **Output** – positive or negative

CRITIC REVIEWS FOR *MOONFALL*

[All Critics \(201\)](#) | [Top Critics \(50\)](#) | [Fresh \(75\)](#) | [Rotten \(126\)](#)

* In the credits for Moonfall, theres at least one astronaut consultant listed, but most of the film feels like a sustained dare to create the least

* This film is totally stupid, but perhaps not stupid enough.

February 24, 2022 | [Full Review...](#)



The Shawshank Redemption (1994)

User Reviews

[+ Review this title](#)

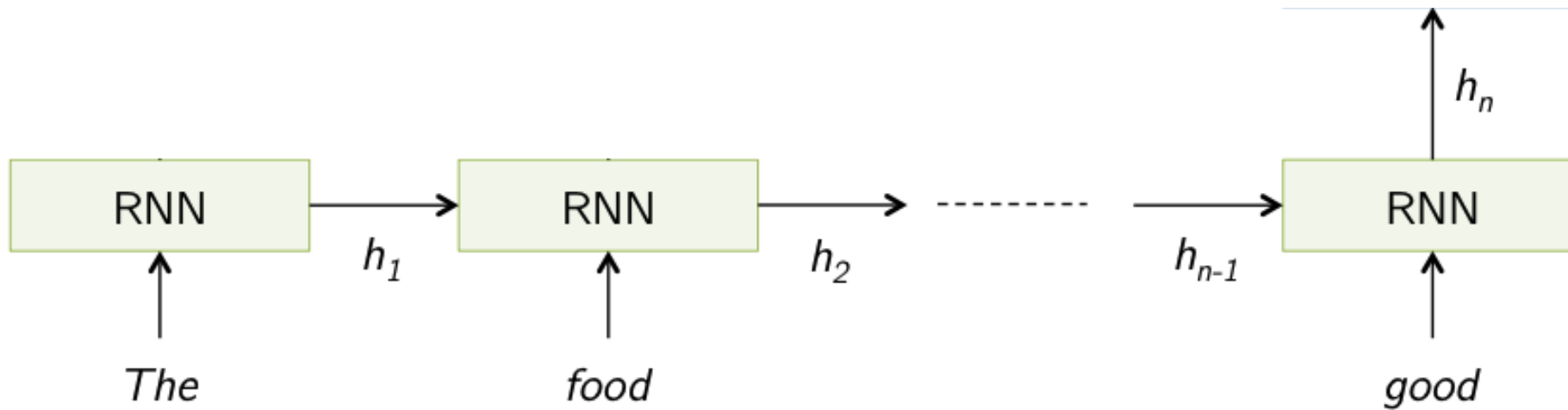
10,147 Reviews

☐ Hide Spoilers Filter by Rating: [Show All](#) [▼](#) Sort by:

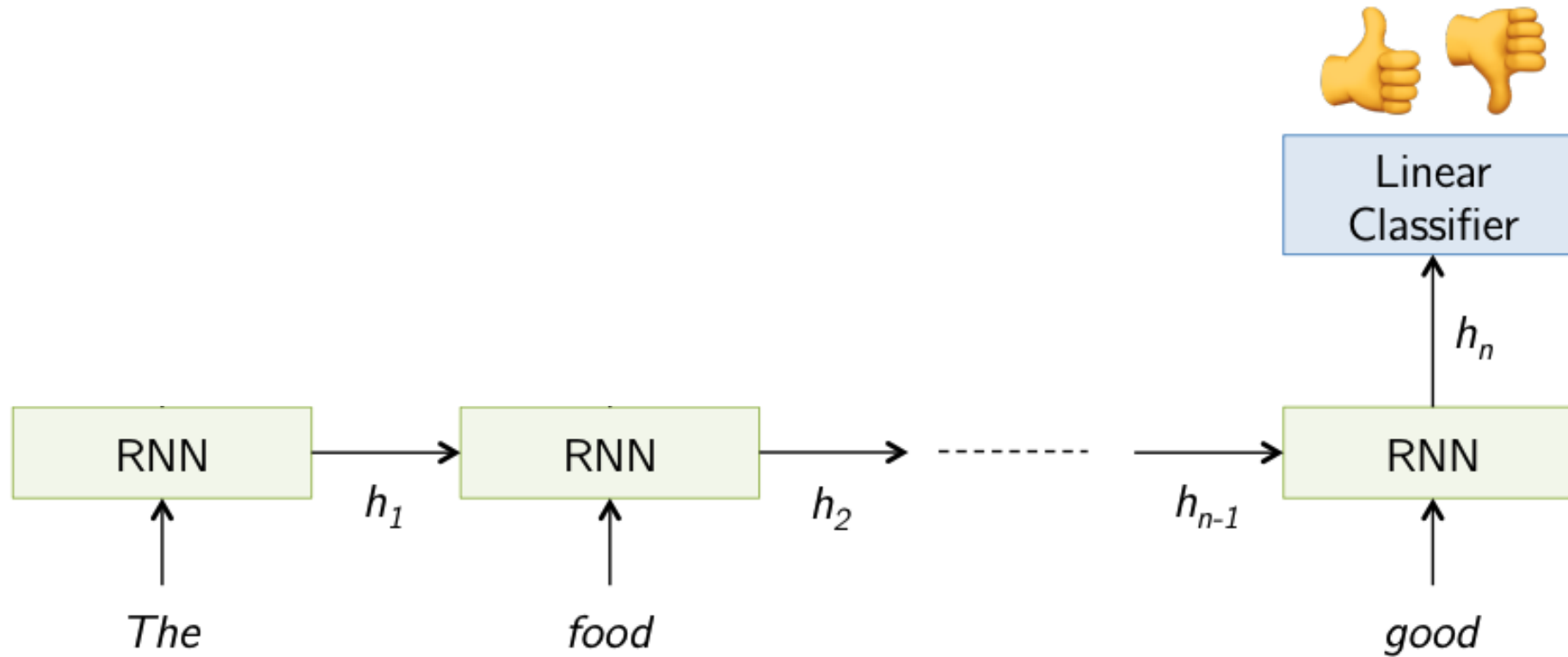
★ 10/10

Tied for the best movie I have ever seen
[carflo](#) 26 November 2003

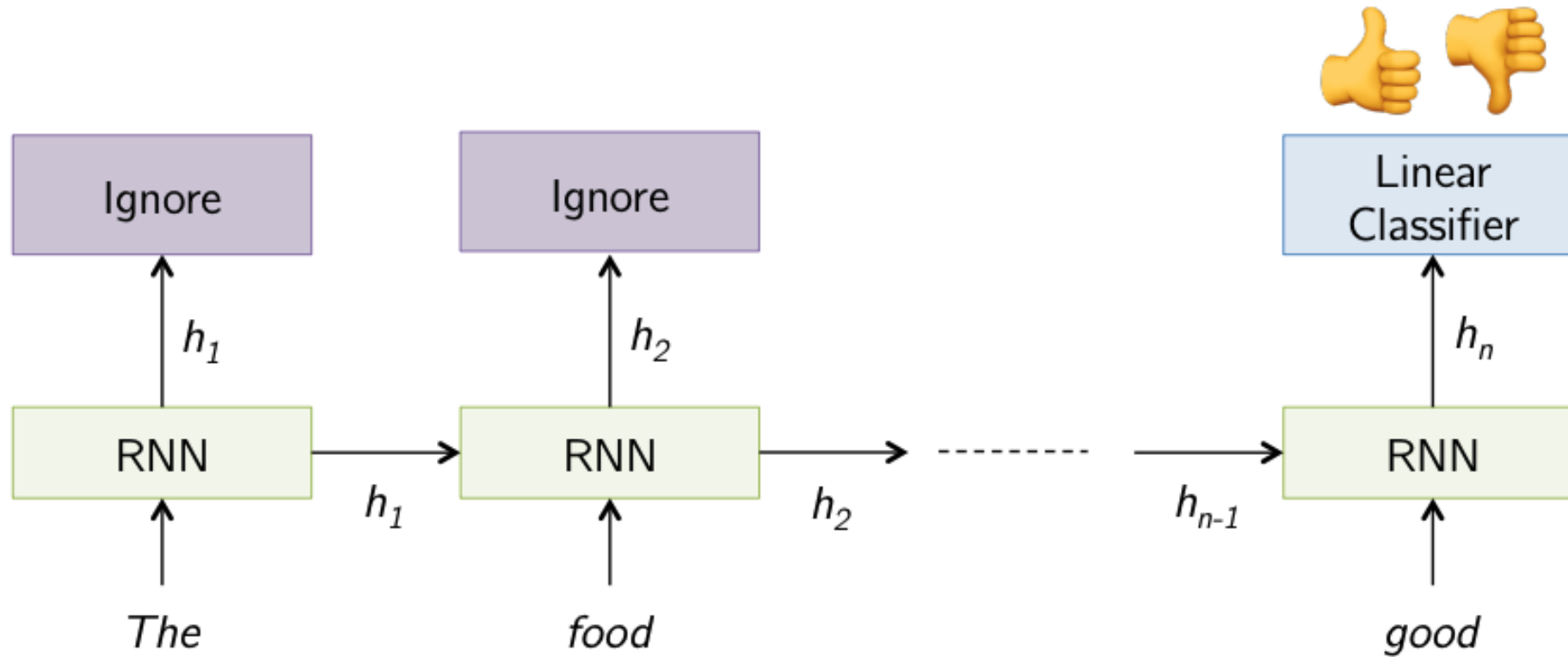
Sentiment Classification



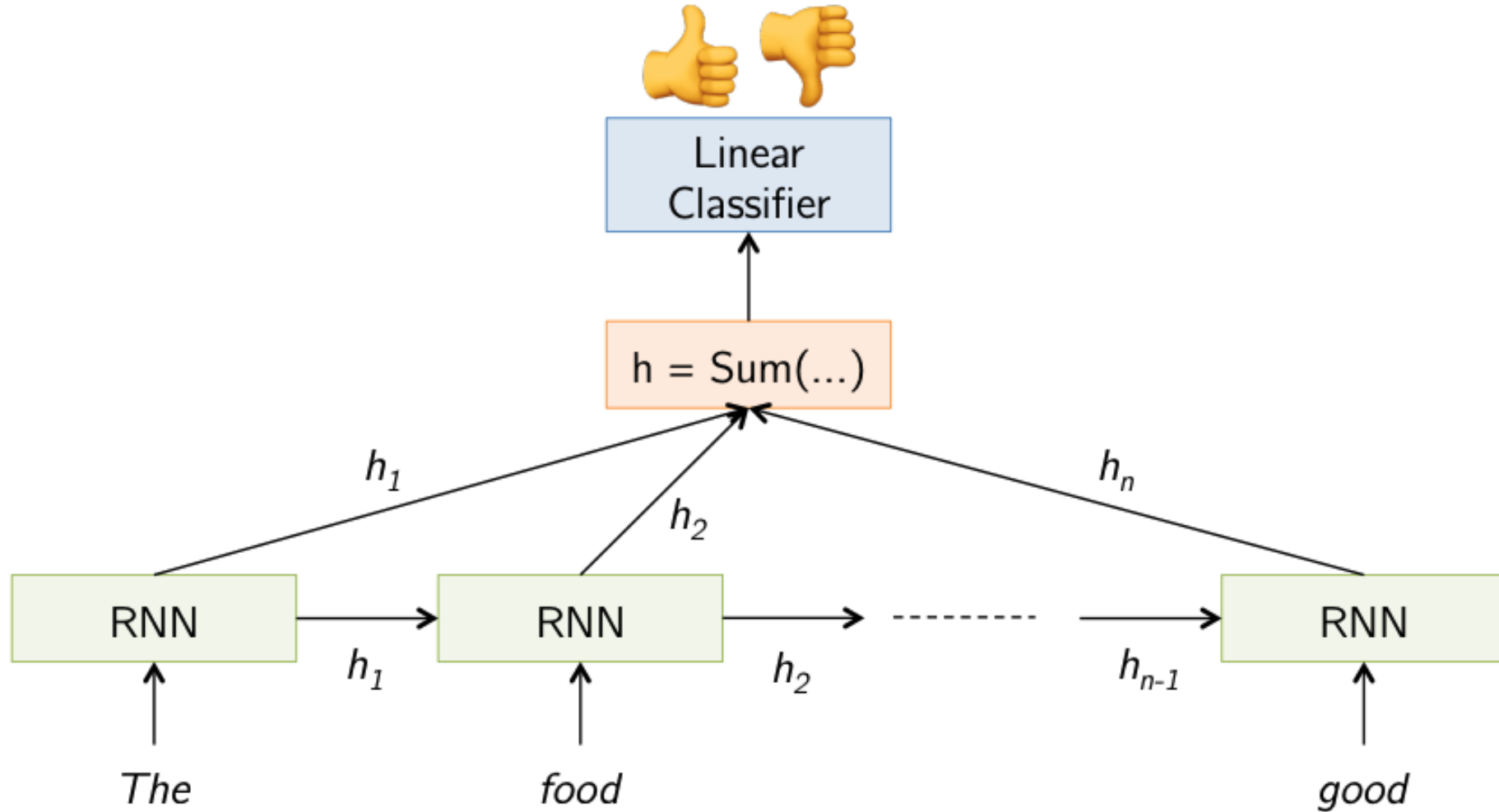
Sentiment Classification



Sentiment Classification



Sentiment Classification



How to train RNN?

Backpropagation Through Time (BPTT)

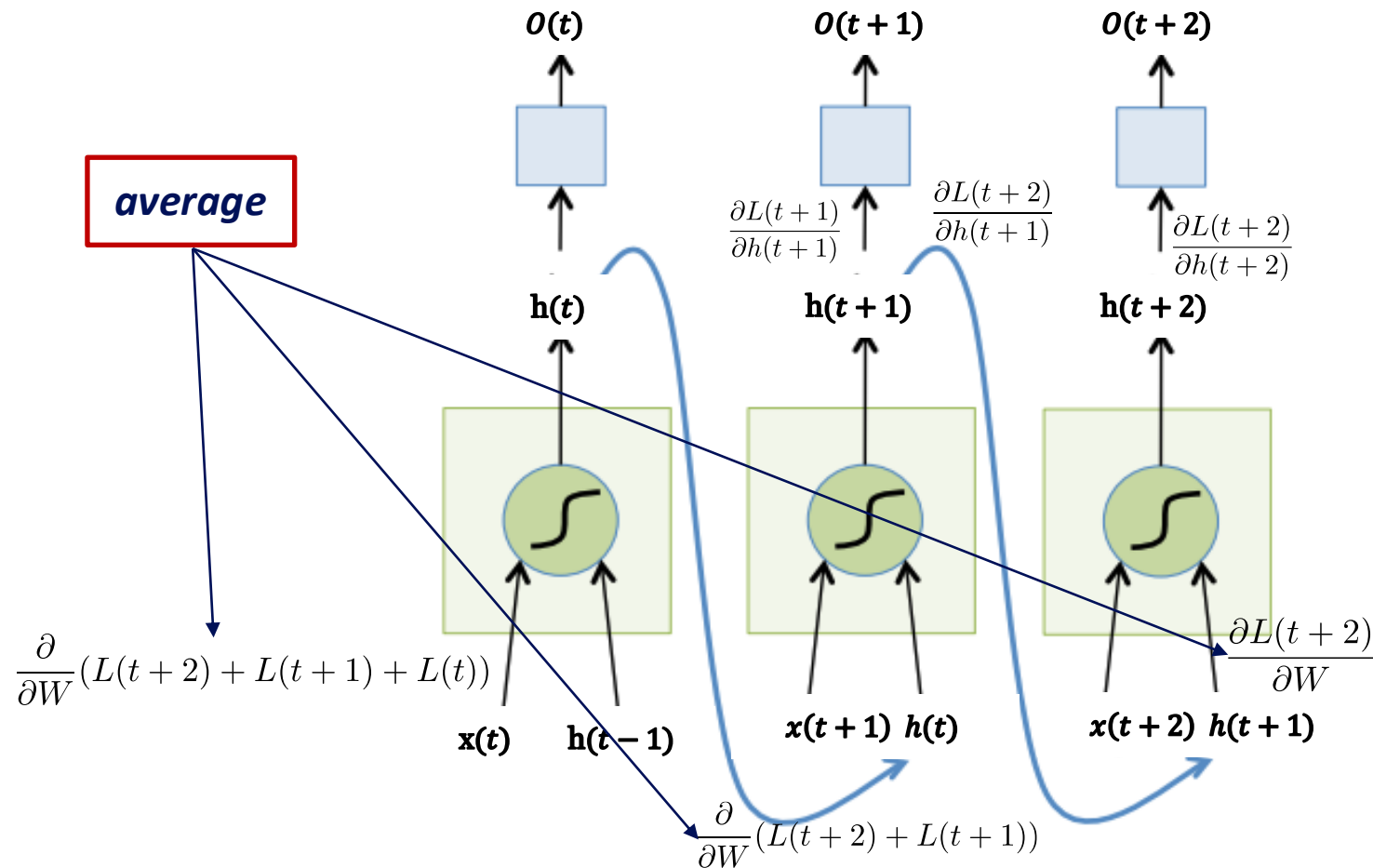
- Unfold (copy) the RNN in time during the forward pass
- Treat the unfolded RNN as a multi-layer NN
- The weight updates are computed for each copy in the unfolded network, then averaged

- Easy, right?

How to train RNN?

Backpropagation Through Time (BPTT)

$$h(t) = \text{sigmoid} \left(W \begin{bmatrix} x(t) \\ h(t-1) \end{bmatrix} \right)$$
$$O(t) = g(h(t))$$



Issue with vanilla RNN and BPTT?

Vanishing or Exploding gradients

- if the largest singular value of W is **small**, then the vanishing gradients problem will occur;
- if the largest singular value of W is **large enough**, then the gradient will explode.

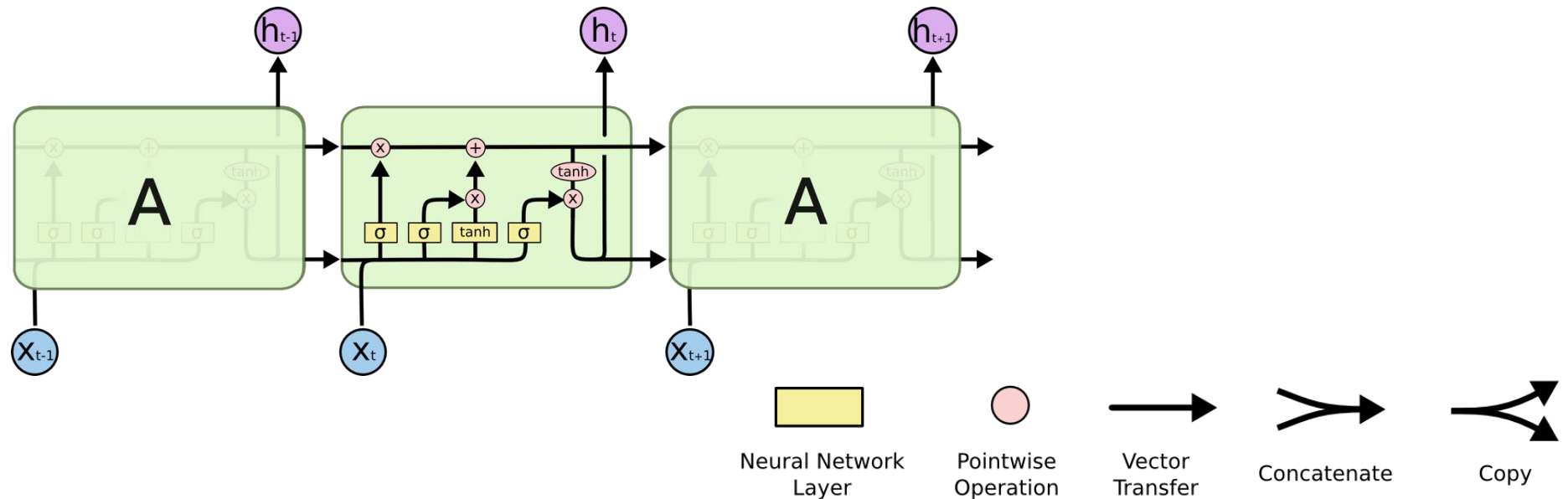
→ Long Short-Term Memory networks – usually just called “LSTMs” – are a special kind of RNN, capable of learning long-term dependencies.

Pascanu, Razvan, Tomas Mikolov, and Yoshua Bengio. "On the difficulty of training recurrent neural networks." In *International conference on machine learning*, pp. 1310-1318. PMLR, 2013.

One Solution - LSTM

Long Short-Term Memory

- key idea: introduce the so-called cell state - the long-term memory
- ***Gated*** RNNs



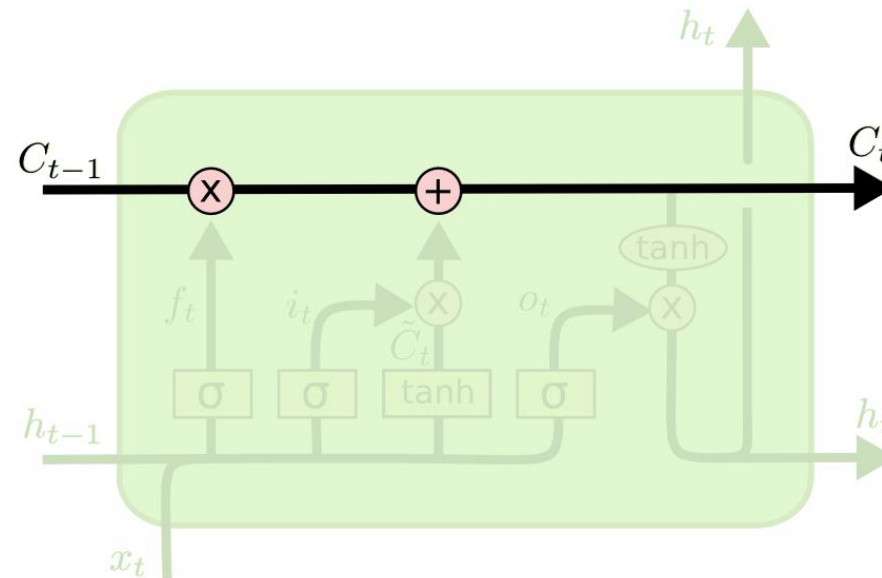
Hochreiter, Sepp, and Jürgen Schmidhuber. "Long short-term memory." *Neural computation* 9, no. 8 (1997): 1735-1780.

You can read more here: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

LSTM: Cell State

Cell state –

- “**Constant Error Carousel**” (CEC) which ensures that gradients don’t decay; why?
- (very) old histories can be forwarded to the current time (see caveats later)



$$\vec{C}(t) = f(t) * \vec{C}(t - 1) + \text{Input}(t)$$

LSTM: Cell State

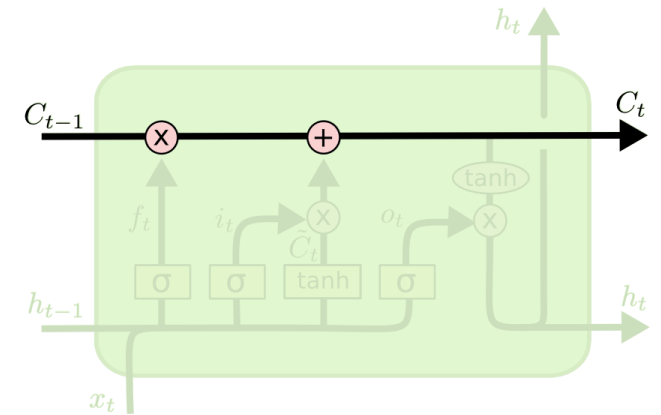
“Constant Error Carousel” (CEC) which ensures that gradients don’t decay; why?

- Ignore the input (not a function of the past)

- Hence,
$$\begin{aligned}\vec{C}(t) &= f(t) * \vec{C}(t-1) = f(t) * f(t-1) * \vec{C}(t-2) = \dots \\ &= f(t) * f(t-1) * \dots * f(t-n+1) * \vec{C}(t-n)\end{aligned}$$

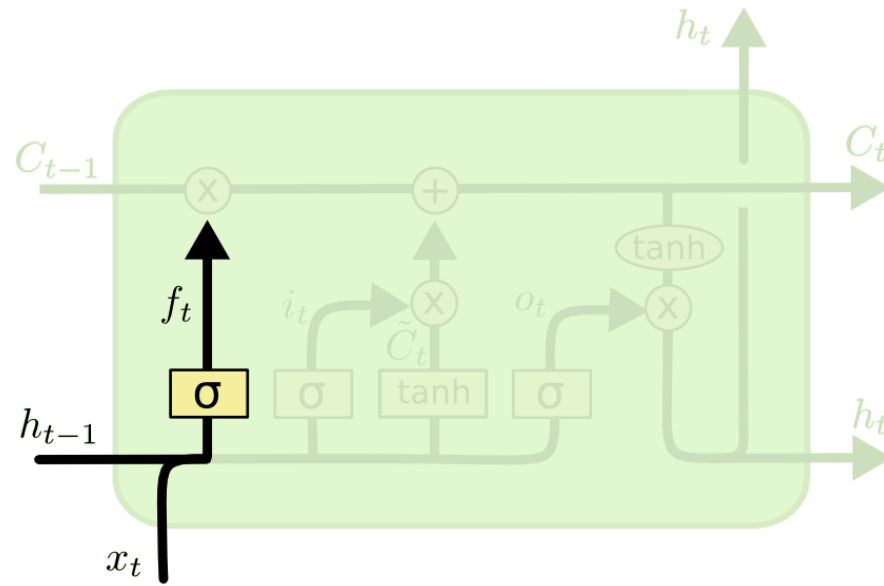
$$\nabla \vec{C}(t) = f(t) * f(t-1) * \dots * f(t-n+1) * \nabla \vec{C}(t-n)$$

- Do you see another problem?



LSTM: Forget Gate

- The current input + the short history (hidden state) to decide the forgetting coefficient
- Why do we use sigmoid here?

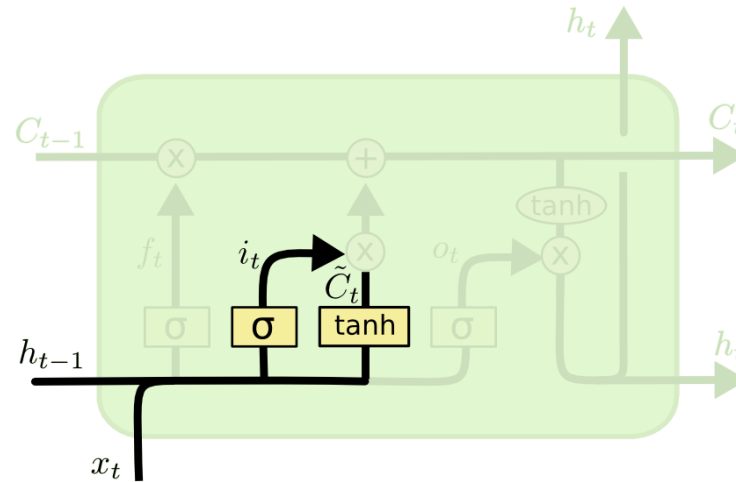


$$\vec{C}(t) = f(t) * \vec{C}(t-1) + \text{Input}(t)$$
$$f(t) = \sigma \left(\mathbf{W}_f [\vec{h}(t-1), \vec{x}(t)] + b_f \right)$$

LSTM: Input Gate

Self-gated - The current input + the short history (hidden state) to **scale** the current input

- Why do we use tanh to transform the input?



$$\text{Input}(t) = i(t) * \tilde{C}(t)$$

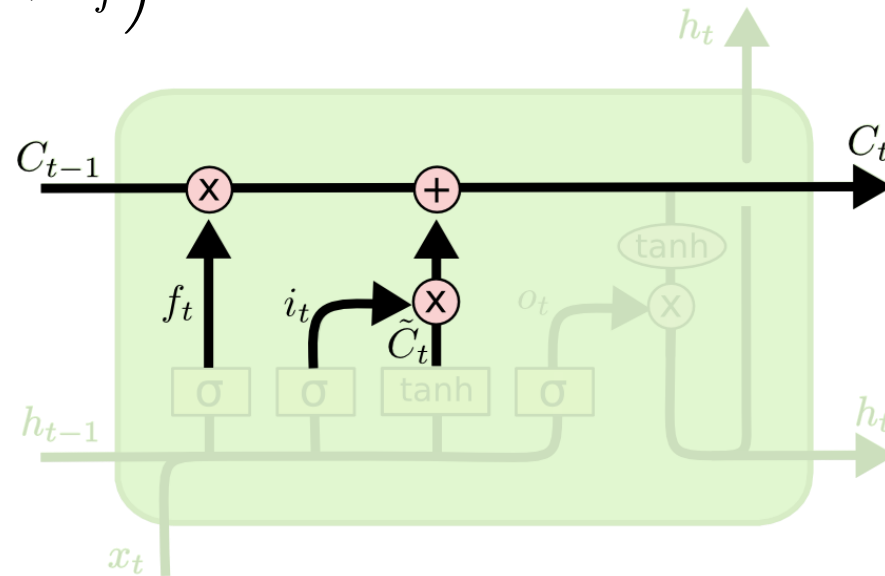
$$i(t) = \sigma \left(\mathbf{W}_i [\vec{h}(t-1), \vec{x}(t)] + b_i \right)$$

$$\tilde{C}(t) = \tanh \left(\mathbf{W}_C [\vec{h}(t-1), \vec{x}(t)] + b_C \right)$$

LSTM: Cell State Update

$$\vec{C}(t) = f(t) * \vec{C}(t-1) + \text{Input}(t)$$

$$f(t) = \sigma \left(\mathbf{W}_f [\vec{h}(t-1), \vec{x}(t)] + b_f \right)$$



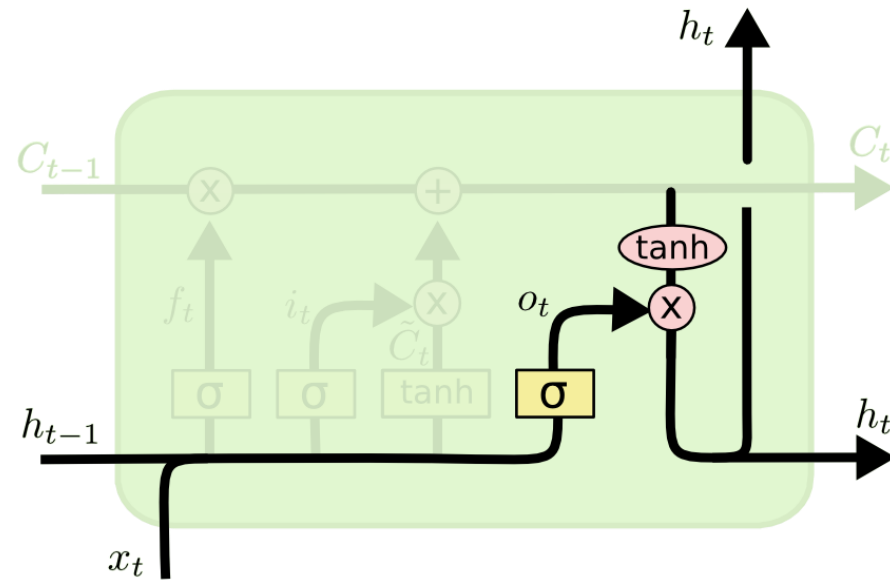
$$\text{Input}(t) = i(t) * \tilde{C}(t)$$

$$i(t) = \sigma \left(\mathbf{W}_i [\vec{h}(t-1), \vec{x}(t)] + b_i \right)$$

$$\tilde{C}(t) = \tanh \left(\mathbf{W}_C [\vec{h}(t-1), \vec{x}(t)] + b_C \right)$$

LSTM: Output Gate

Self-gated - The current input + the short history (hidden state) to **scale** the current cell state



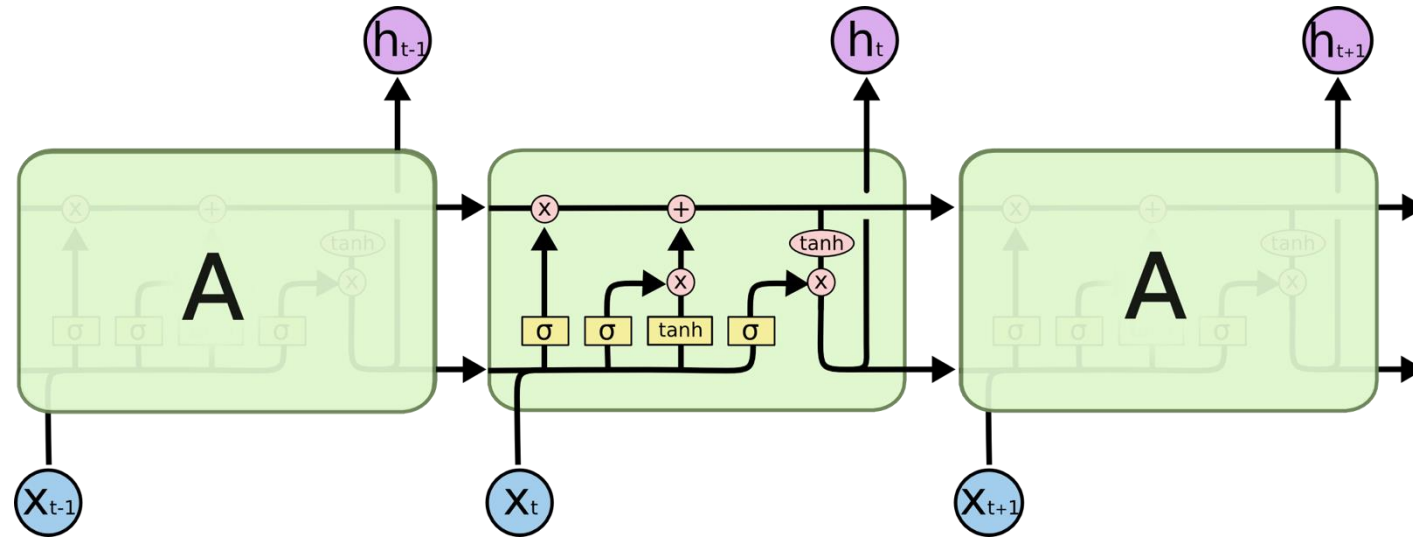
$$o(t) = \sigma \left(\mathbf{W}_o [\vec{h}(t-1), \vec{x}(t)] + b_o \right)$$

$$\vec{h}(t) = o(t) * \tanh(\vec{C}(t))$$

LSTM: Assembly..

$$\vec{C}(t) = f(t) * \vec{C}(t-1) + \text{Input}(t)$$

$$f(t) = \sigma \left(\mathbf{W}_f[\vec{h}(t-1), \vec{x}(t)] + b_f \right)$$



$$\text{Input}(t) = i(t) * \vec{C}(t)$$

$$i(t) = \sigma \left(\mathbf{W}_i[\vec{h}(t-1), \vec{x}(t)] + b_i \right)$$

$$\vec{C}(t) = \tanh \left(\mathbf{W}_C[\vec{h}(t-1), \vec{x}(t)] + b_C \right)$$

$$o(t) = \sigma \left(\mathbf{W}_o[\vec{h}(t-1), \vec{x}(t)] + b_o \right)$$

$$\vec{h}(t) = o(t) * \tanh(\vec{C}(t))$$

LSTM: Choice of Activations

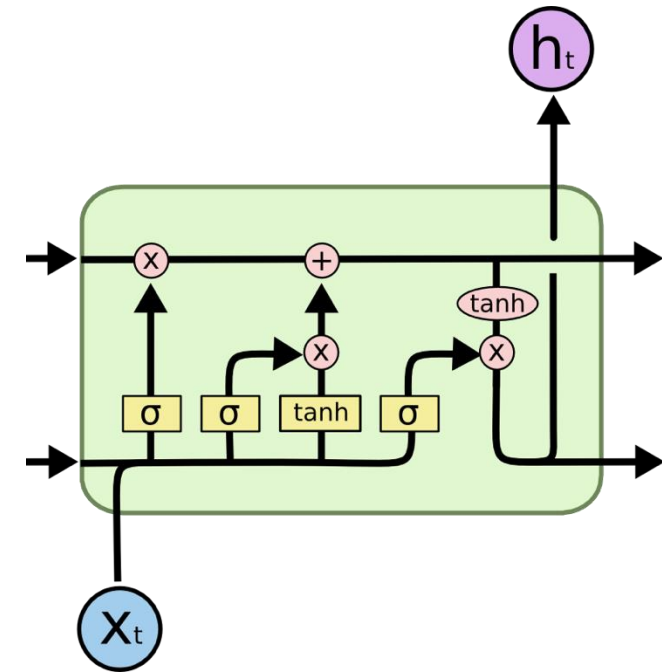
Question: can we replace tanh by ReLU in the input gate? Is it safe?

- The input scaler $i(t) \in (0, 1)$
- $\tilde{C}(t)$ computed from ReLU can explode...
- The same holds for the output gate

$$\text{Input}(t) = i(t) * \tilde{C}(t)$$

$$i(t) = \sigma \left(\mathbf{W}_i [\vec{h}(t-1), \vec{x}(t)] + b_i \right)$$

$$\tilde{C}(t) = \tanh \left(\mathbf{W}_C [\vec{h}(t-1), \vec{x}(t)] + b_C \right)$$



LSTM: Caveat

Question: can we use it on a really long sequence/series?

$$\nabla \vec{C}(t) = f(t) * f(t-1) * \dots * f(t-n+1) * \nabla \vec{C}(t-n)$$

- The forgetting scaler $f(t) \in (0, 1)$
- What if $f(t) = 0.99, n = 1000$?

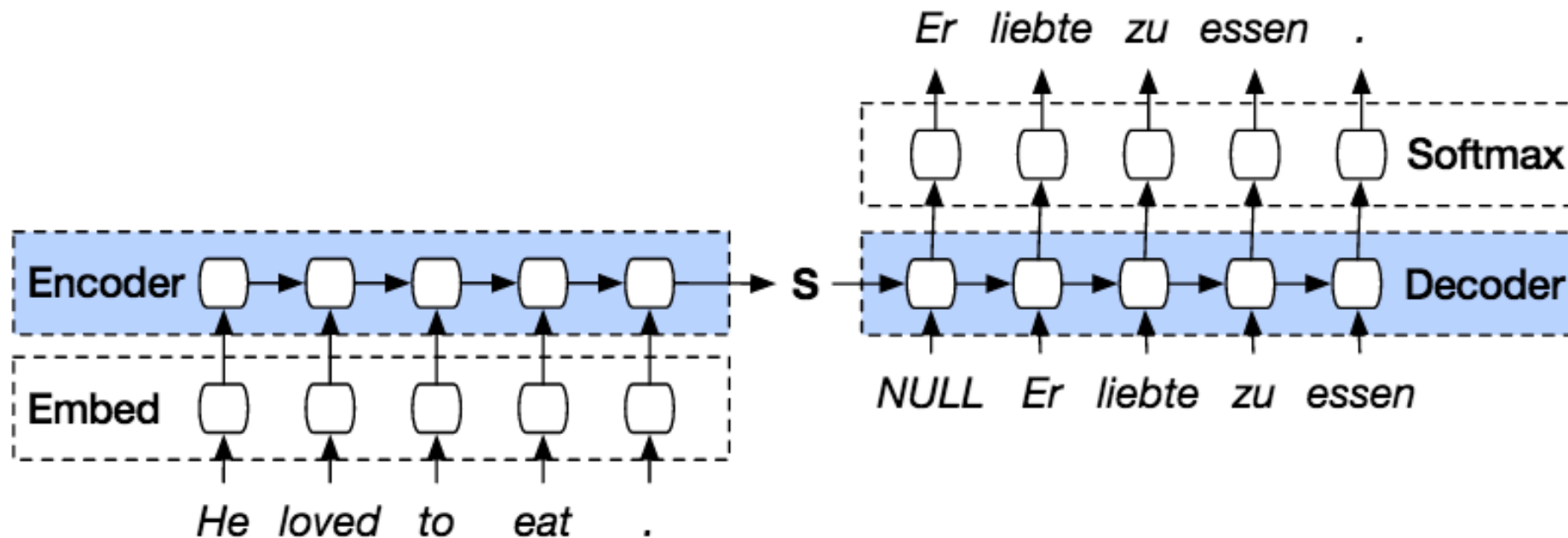
$$0.99^{1000} \approx 4.317 \times 10^{-5}$$

- Modelling very very long sequences?
 - **Attention?** Attention!

Beyond LSTM – Seq2seq model

Sequence-to-sequence

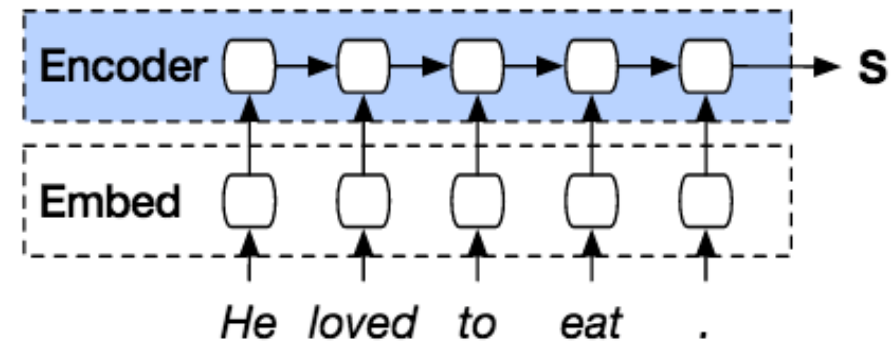
- (arguably) the most classical model proposed in *natural language processing*
- Machine translation, time series forecasting (multi-step), ...



Beyond LSTM – Seq2seq model

- Encoder – everything you have learned...
 - Word embedding: typically, a fully connected NN/multilayer NN
 - The initial hidden state $\vec{h}(0)$ is usually randomized

$$\begin{aligned}\vec{x}(t) &= \text{Embedding}(\vec{w}(t)) \\ \vec{h}_E(t) &= \text{RNN}_E(\vec{x}(t), \vec{h}_E(t-1)) \\ \vec{s} &= \vec{h}_E(\text{last})\end{aligned}$$



Beyond LSTM – Seq2seq model

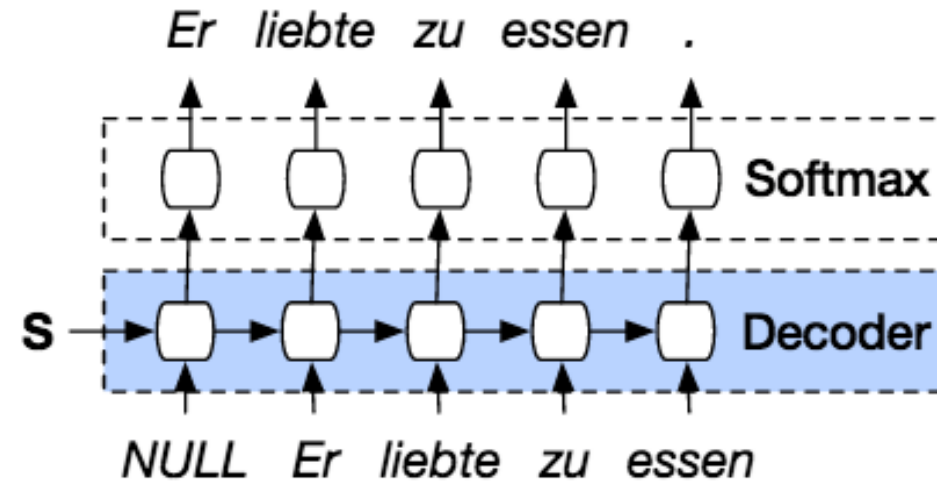
Decoder

- take $\vec{s} = \vec{h}_E(\text{last})$ to initialize decoder's hidden state
- Input is the decoded/predicted symbol from the previous step

$$\vec{h}(0) = \vec{h}_E(\text{last})$$

$$\vec{h}_D(t) = \text{RNN}_D(O(t-1), \vec{h}_D(t-1))$$

$$O(t) = \text{Softmax}(\vec{h}_D(t))$$





Thank you – Any questions?



Universiteit
Leiden
The Netherlands

e.raponi@liacs.leidenuniv.nl

Discover the world at Leiden University